Kafi Mobile App — Production-Readiness Audit

Nanny · Family · Cross-role · Backend — analysis only

Kafi Mobile App — Production-Readiness Audit

Platform: Flutter app (`kafi_app/`) — nanny role, family role, and nanny↔family cross-role flows, plus the backend (Cloud Functions / Firestore rules / indexes) where it breaks the app. **Method:** multi-auditor code trace. Every finding is anchored to `file:line`, rated by severity, and stated as a concrete **failing use case**. **Date:** 2026-07-21 · **Mode:** analysis only (no code changed).

How to read this

Each finding is `[SEVERITY] title — file:line — Failing use case — Fix`.

- **CRITICAL** — blocks launch: data loss, wrong-recipient messaging, no revenue, store rejection, or the core matchmaking handshake failing.
- **MAJOR** — a real feature is broken or a flow silently fails for a common case.
- **MINOR** — correctness/UX/validation gap with a narrow blast radius.
- **CONFIRMED** = traced in code end-to-end. **SUSPECTED** = depends on runtime/deploy data that can't be executed here.

Config baseline that shapes several findings: `AppConfig.useMock = false` (live Firebase) but `AppConfig.useMockSubscription = true` (`config/app_config.dart:9,13`) — so **the entire paywall/subscription layer runs on the mock in the shipping build**, and the `.firebaserc` default project is `yhgc-testing`.

Executive summary

The app's happy paths are largely well-built and defensively coded (consistent try/catch, optimistic-message rollback, error/empty/retry states on most list screens). The production risk is concentrated in a handful of **single-cause, high-impact** defects:

- **6 Critical** — account deletion is fully broken; there is no real payment path (every plan is free); a nanny's "Message Family" opens the **wrong** conversation; the nanny is never told she's hired; ending a hire is invisible to the other side; and the scheduled subscription jobs crash on a missing index.
- **~16 Major** — trial-based access never actually unlocks; a security bypass grants free access in the shipped project; nanny trial/chat screens cross-wire when a nanny has two families; subscription expiry never re-checks mid-session; several notifications misfire or never fire.
- **~20 Minor** — validation, i18n, resume-routing, and rules-hardening gaps.

Three systemic root causes run through most Majors/Criticals (see §5): (1) **no realtime for hires/trials/employment** — only chat threads stream, so any state that doesn't mutate the thread doc goes stale; (2) **family-oriented navigation/bindings reused for the nanny** without role-branching — the source of the wrong-thread and trial cross-wire bugs; (3) **notification triggers assume a single actor** and several key events have **no trigger at all**.

---

1. Nanny role

Critical

[CRITICAL] "Message Family" opens the wrong conversation (or creates a self-thread) — CONFIRMED — `controllers/chat_controller.dart:285-325`, `views/nanny/application_detail_screen.dart:148,166,179`, `utils/app_navigation.dart:54-71` Failing use case: The nanny CTA calls `openChat(nannyId: app.nannyId)` — but `app.nannyId` is *her own* id. `openThreadForNanny` then sets `familyId = currentUserId` (herself) and matches `threads.firstWhereOrNull((t) => t.nannyId == nannyId)`. Since **every** thread a nanny has satisfies `t.nannyId == her own id`, it opens the **first** thread. A nanny engaged with Families A and B who taps "Message Family" on B's application lands in **A's** chat and can send B's message to A. If no thread exists yet, `findOrCreateThread(familyId: ownId, nannyId: ownId)` creates a nonsensical self-thread. Fix: branch by role — for a nanny, resolve the thread by `familyId`, never by her own `nannyId`.

Major

[MAJOR] Trial "View trial" cross-wires to the wrong trial for a 2-trial nanny — proof uploaded against the wrong family — CONFIRMED — `views/family/chat_screen.dart:770-793`, `controllers/trial_controller.dart:66-67,116-136` Failing use case: the chat trial banner does `Get.toNamed(Routes.trial)` with **no trialId**; TrialScreen shows `displayed = selected ?? active`, and a nanny's `active` is the *first* active trial. The app's own cap allows 2 concurrent trials, so opening "View trial" from Family B's chat renders **Family A's** trial + day-proof grid — she can upload today's childcare proof against the wrong family's record (privacy leak). Fix: pass the thread's `trialId` and open by id.

[MAJOR] Nanny's Trial screen labels the "Family" card with the nanny's own name — CONFIRMED — `views/family/trial_screen.dart:255-256,333` Failing use case: the left "Family · location" card uses `famName = currentUser.fullName`; when the nanny views the trial, `currentUser` is *her*, so the card meant to identify the family shows her own name and the family's real name is never shown. Fix: resolve family identity from `trial.familyId`/thread.

[MAJOR] Opening the trial route as a nanny runs `FamilyBinding` and pollutes the session — CONFIRMED — `config/routes.dart:132`, `bindings/family_binding.dart:15-33`, `utils/app_navigation.dart:54-71` Failing use case: `Routes.trial` is bound to `FamilyBinding`, which `Get.put`s `FamilyShellController` + browse/profile/jobs controllers. Once a nanny taps "View trial", a permanent `FamilyShellController` exists, so `AppNavigation.openChat` then takes the family-shell branch and the nanny's chat-entry buttons become dead; family-only controllers also run `onInit` under a nanny account. Fix: give `Routes.trial` a role-agnostic/no binding, or guard the FamilyBinding puts by role.

[MAJOR] False "Session expired" error on every deliberate logout / account-deletion — CONFIRMED — `services/session_monitor.dart:60-83`, `views/family/settings_screen.dart:506-509` Failing use case: `signOut()`/`user.delete()` emits `authStateChanges(null)` while the route is still `/nanny-home` or `/delete-account` (neither is in `_noUserRoutes`), so `_handleSessionExpired()` fires the amber "Session expired – please sign in again" snackbar and re-navigates to welcome, then the settings/delete handler navigates again — an alarming error + double redirect on every normal logout. Fix: set an `intentionalSignOut` flag SessionMonitor checks, or navigate to welcome *before* `logout()`.

[MAJOR] No availability control — a hired/on-trial nanny still matches as "Available now" — CONFIRMED — `views/nanny/nanny_dashboard_screen.dart`, `services/match_service.dart:197-208` Failing use case: `availability` is set only in onboarding; nothing ever assigns `onTrial` or marks a hired nanny unavailable (grep: zero `AvailabilityStatus.onTrial` writes), and the dashboard has no toggle. A mid-trial or already-hired nanny keeps `availableNow` (→ availability score 1.0) and keeps surfacing as fully available in browse/smart-match, with no way to pause her listing. Fix: dashboard availability toggle + auto-set on trial-accept/hire.

Minor

- **NAN-1 · 90-day inactivity auto-logout never fires** — `session_monitor.dart:30-54`. `onInit` re-stamps `lastActivityAt=now` on every launch before checking, so a nanny returning after 90+ days is never logged out (spec §21 defeated). *Fix:* evaluate inactivity before touching on launch; wire `touch()` to a routing observer.
- **NAN-2 · Resume routing skips the Experience & References steps** — `auth_controller.dart:335-350`. A nanny who closes the app on the Experience step is resumed on Documents, silently bypassing Experience/References (thereafter only reachable via Settings). *Fix:* extend the resume ladder to exp/refs.
- **NAN-3 · `totalExperienceYears` uses calendar-year deltas** — `nanny_model.dart:391-394`. `toDate.year - fromDate.year` counts an 11-month job inside one year as 0 years but a 2-month job across New Year as 1, so her experience-gated match % is materially wrong. *Fix:* compute months/days.
- **NAN-4 · Admin-`rejected` documents survive a resubmit** — `nanny_profile_controller.dart:1025-1032`. On resubmit only `uploaded→reviewing` is remapped; a doc the admin marked `rejected` (not re-picked) stays `rejected` while overall status flips to `pending`. *Fix:* reset non-missing `rejected` docs to `reviewing` on resubmit.
- **NAN-5 · Several nanny notification types are dead taps** — `notification_controller.dart:211-218`. For legacy/no-`route` notifications, `documentsRejected`/`documentsApproved`/`profileVerified`/`hired`/`profileViewed` fall through to a no-op `return`, so tapping "You've been hired" goes nowhere. *Fix:* route these types to the right screen.
- **NAN-6 · OTP entry has no backspace-to-previous, paste, or SMS autofill** — `otp_verify_screen.dart:234-238`. Mistype + backspace on an empty box doesn't move focus, and there's no autofill/auto-submit — core-flow friction. *Fix:* wire focus-back + `autofillHints` + auto-retrieval.
- **NAN-7 · Splash can hang if Firestore stalls after auth resolves** — `auth_controller.dart:69-87,305-330`. Only `getCurrentUser()` is timeout-guarded; if Firestore stalls in `_routeForUser`, `bootstrapStartup` never returns → splash spins forever with no retry. *Fix:* timeout-guard `_routeForUser` → `startupError`.
- **NAN-8 · Job-detail "Apply" gives no already-applied signal** — `job_detail_screen.dart:332-349`. Apply always routes to smart-match; the duplicate guard only fires after the nanny finishes smart-match + the cover sheet — wasted effort (the write is correctly blocked). *Fix:* show already-applied state on the button.

---

2. Family role

Critical

[CRITICAL] "Delete Account" always fails — nothing is ever deleted — CONFIRMED — `services/firebase/firebase_auth_service.dart:203,221`, `firestore.rules:52` (+ no `deletionAudits` rule) Failing use case: `deleteAccount()` first does an **awaited, un-caught** write to `collection('deletionAudits')`, which has **no security rule** → default-denied → the method throws before `user.delete()` (line 217) ever runs; and `_users.doc(uid).delete()` (line 221) is `isAdmin()`-only and also denied. So the `onUserDeleted` cascade never fires and the user + all data persist. This is an **App Store 5.1.1(v) rejection** and a data-rights violation. Fix: add a `deletionAudits` create rule (or write it from a callable), wrap each best-effort write in its own try/catch, call `user.delete()` first with `requires-recent-login` → OTP re-auth, and run the cascade via an authenticated callable using the admin SDK.

[CRITICAL] No real payment integration — every plan is granted for free; and the prod path is rules-denied — CONFIRMED — `config/app_config.dart:13`, `services/mock/mock_subscription_service.dart:62-76`, `services/firebase/firestore_subscription_service.dart:81-92`, `firestore.rules:89-91` Failing use case: there is **no `purchases_flutter`/RevenueCat/IAP dependency anywhere** (only a `revenueCatId` field). With `useMockSubscription=true`, tapping any plan writes `status:active` to SharedPreferences — **no money is charged**, the family gets unlimited access for free. If the flag is flipped to prod without a server path, `subscribe()` writes `families/{id}.subscription` directly, which the family-update rule **denies** — so no one could ever subscribe either. Fix: implement the RevenueCat purchase → webhook → entitlement path; keep the client write locked; don't ship the paywall as the real gate until then.

Major

[MAJOR] "Restore Purchases" is a no-op — CONFIRMED — `controllers/subscription_controller.dart:169-178`, reached from `settings_screen.dart:138-144` Failing use case: a paying family reinstalls and taps Restore Purchases; it reports success (comment says "Call RevenueCat restore in production" but the body only calls `refreshAndEnforce()`, which in mock reads now-empty local prefs) — access stays free. Fix: implement `Purchases.restorePurchases()` and re-derive entitlement from the store/webhook.

[MAJOR] Subscription expiry/renewal is never re-checked on resume — CONFIRMED — `controllers/subscription_controller.dart:63-65` Failing use case: the doc says "called on app start, foreground resume, and CF trigger" but the only callers are `onInit`/`subscribe`/`restore` — there is **no `WidgetsBindingObserver`/`AppLifecycleState` anywhere in `lib/`**. When a subscription expires/renews while the app is open, contacts/chat stay in their prior state until a full restart, so `contactsHidden`/`chatLocked`-on-expiry never fires mid-session. Fix: add a lifecycle observer calling `refreshAndEnforce()` on resume (ideally a Firestore listener on the family's `subscription`).

[MAJOR] Expiry-driven chat lockdown is disabled in the shipping config — CONFIRMED — `controllers/chat_controller.dart:116` Failing use case: `_skipSubscriptionGates = AppConfig.subscriptionUsesMock` (= true), which short-circuits every gate (`openThread`, `sendCurrent`, `onSubscriptionLocked`, …). When a mock-subscribed family lapses, chat isn't force-closed or paywalled client-side; correctness depends entirely on server rule state the app can't see. Fix: decouple the lockdown UX from the mock flag; verify `syncMockSubscription` clears `mockDev` on expiry; add an expired-family chat test.

[MAJOR] A family's second job post can never be fully edited — CONFIRMED — `controllers/family_profile_controller.dart:96-98,244-247`, `services/firebase/firestore_job_service.dart:100-103` Failing use case: the edit flow hydrates/saves via `posts.first`, and `getJobsByFamily` has no `orderBy` (so "first" = lowest doc-id, not "most recent"). A family with 1 full-time + 1 part-time post can only ever edit the first-id job's roles/duties/benefits/visa/type/trial — the other job's full details are uneditable. Fix: pass the target job id into the edit flow; order `getJobsByFamily` by `createdAt`.

[MAJOR] Compare only ever shows the first two shortlisted nannies — CONFIRMED — `views/family/compare_screen.dart:20` Failing use case: `shortlistedNannies.take(2)` with no picker — a family that shortlists 5 can only compare the two oldest entries; no other pair is comparable. Fix: add a two-nanny picker feeding the compare row.

Minor

- **FAM-1 · OTP client timer rejects still-valid codes** — `auth_controller.dart:181-184`. A correct code entered just after the 5-min local timer hits 0 is rejected as expired, though Firebase would still accept it. *Fix:* let Firebase reject expired codes; drop the client pre-check.
- **FAM-2 · Subscribers' viewed profiles aren't recorded → "re-locked" never applies** — `subscription_controller.dart:133`, `browse_screen.dart:114-119`. After a subscribed family expires, previously-unlocked nannies open as "locked/new" instead of "re-locked" (cosmetic). *Fix:*

track viewed ids for subscribers too.

- **FAM-3 · Pricing screen + grace/expiry banners hardcoded English** — `pricing_screen.dart:45,105,157,222-227`, `browse_screen.dart:174-176`. An Arabic-locale family sees Pricing and the payment-issue banner in English only. *Fix:* move to `AppStrings`/`.tr`.
- **FAM-4 · My-Jobs quick-edit accepts salary min > max** — `my_jobs_screen.dart:417-426`. Min 5000 / Max 1000 saves an inverted range that renders oddly to nannies (the main form validates this). *Fix:* run `Validators.salaryRange` before save.
- **FAM-5 · "Post New Job" pre-fills from the existing post; a same-type repost is silently blocked** — `family_binding.dart:27`, `family_profile_controller.dart:248-255`. Tapping "Post New Job" pre-populates the existing post and submitting fails with `familyJobTypeLimit`, which reads as a bug rather than an intended 1-FT/1-PT cap. *Fix:* reset the form for a fresh post; surface the cap proactively.
- **FAM-6 · Opening an applicant ignores the free-view result** — `family_applicants_screen.dart:258-273`. An over-limit free family taps an applicant and still reaches the locked profile with no "no free views left" nudge (browse does nudge). *Fix:* honor `recordViewIfAllowed`'s bool → route to pricing.
- **FAM-7 · Contact reveal shows only "…" for up to 15s, no spinner/retry** — `profile_unlocked_screen.dart:131-135`, `firestore_user_service.dart:189-213`. If the reveal function is cold/slow, the family stares at "…" then "unavailable" with no retry. *Fix:* spinner + inline retry.
- **FAM-8 · `resolveNannyCard` silently falls back to a demo nanny** — `nanny_card_resolver.dart:24`. Any future deep-link nav to a profile screen without the card object shows a fabricated seed nanny to a real user (no live trigger today). *Fix:* return an explicit empty/error card + guard the profile screens.
- **FAM-9 · "Arabic" filter keys off the first-3-languages tag only** — `firestore_job_service.dart:54-63`, `nanny_card_model.dart:61`. A nanny who lists Arabic as her 4th+ language is excluded from the Arabic filter. *Fix:* filter on the full `languages` list.

---

3. Cross-role (nanny ↔ family) integrity

Critical

[CRITICAL] The nanny is never notified she was hired — no push, inbox, chat message, or realtime card update — CONFIRMED — `functions/src/triggers/stats.ts:80-88`, `utils/notifications.ts:83`, `controllers/notification_controller.dart:215`, `controllers/nanny_profile_controller.dart:213-227`
Failing use case: the family taps "Hire" → `_createHireFromTrial` writes `hires/{id}`; the only trigger, `onHireCreated`, **only increments `stats.hiresCount`**. No chat message is posted; `setOutcome` merely flips the thread's trial badge to `completed` (so the green trial pill *vanishes* with nothing replacing it); the nanny's dashboard hire card is a one-shot load; and the `hired` inbox type/tap-handler exist but **nothing ever writes a `hired` inbox doc**. Net: a nanny is hired and gets **zero** in-app signal. Fix: `onHireCreated` → `writeInbox(nannyId,'hired',…)` + push, and post a system chat bubble on hire.

[CRITICAL] Ending a hire is invisible to the counterparty, and the reason can be silently overwritten — CONFIRMED — `controllers/chat_controller.dart:51-69`, `services/firebase/firestore_hire_service.dart:42-54`
Failing use case: there is **no `onHireEnded` trigger**, **no hire stream** (both `getHiresFor*` are one-shot `.get()`), and `endActiveHire` touches only the `hires` doc. When the family terminates while the nanny is on her dashboard/chat, her "Hired" pill, header badge, hire banner (with a live **Resign** button), and dashboard resign card all stay live for the whole session; if she then taps Resign, `endHire` does a blind `.update` that **overwrites `endReason: terminated → resigned`** and re-stamps `endedAt`, corrupting the record of who ended it. Fix: add `onHireEnded` notify+inbox to the counterparty, stream hires (or write the

terminal state onto the thread doc), and guard `endHire` against re-ending a non-active hire.

Major

[MAJOR] Trial-based access never actually unlocks chat/contact — `activeTrialNannyIds` is only written when a trial ENDS — CONFIRMED — `functions/src/triggers/trial.ts:113-116`; consumed at `firestore.rules:42-45,156-158,181-184`, `functions/src/triggers/contact.ts:22-24` Failing use case: a non-subscribed family gets a trial *accepted* by a nanny but still can't open a chat, send a message, or reveal contact — the only writer of `families.activeTrialNannyIds` is `onTrialEnded` (fires on completed/cancelled/declined); no trigger adds the nanny on pending→accepted or accepted→active, and the family is forbidden by rules from writing the field. So `hasActiveTrialWith`/`isContactEntitled` see an empty array for a family's first trial. (Masked today only because the `yhgc-testing` project lets families self-grant a sub — see §4.) Fix: on trial → `accepted`/`active`, union the nanny into `activeTrialNannyIds`.

[MAJOR] `onTrialResponse` always notifies the *family* and attributes every change to the *nanny* — counter responses misfire — CONFIRMED — `functions/src/triggers/trial.ts:51-87`, `controllers/trial_controller.dart:382-422` Failing use case: the family also mutates status (`acceptCounter`, `declineCounter`), but the trigger unconditionally notifies `after.familyId` with nanny-attributed copy. So when a family accepts the nanny's counter offer, the **nanny gets nothing** and the family gets a self-notification "Nanny accepted your offer!"; declining a counter is likewise silent to the nanny. Fix: branch on who transitioned; route counter-accept/decline to the nanny.

[MAJOR] Trial cancellation notifies nobody — while the UI promises "Both parties will be notified" — CONFIRMED — `views/family/trial_screen.dart:667`, `controllers/trial_controller.dart:608-619`, `functions/src/triggers/trial.ts:118-125` Failing use case: cancelling sets status `cancelled`; `onTrialResponse` has no `cancelled` branch and `onTrialEnded` only notifies when `status==='completed'`, so the counterparty is never told and believes the trial is still on. Both directions are silent. Fix: notify the counterparty on `cancelled`.

[MAJOR] Nanny and family see different match % for the same pair — CONFIRMED — `views/family/family_applicants_screen.dart:239,272`, `views/nanny/job_detail_screen.dart:26`, `services/firebase/firestore_application_service.dart:70-92` Failing use case: the family view uses the frozen apply-time `app.matchScore`; the nanny view recomputes live from her current profile. A nanny who applies at 62% then edits her profile shows 78% on job-detail while the family's applicants list still shows 62% for the identical pair — violating the MatchService "identical everywhere" contract (`match_service.dart:8-10`). Plus browse scores *with* household context and applicants *without* it, so even the family sees two different numbers (e.g. 88% in browse, 71% in applicants). Fix: recompute on read (or refresh the stored score when the nanny profile changes); converge on one scoring context.

[MAJOR] Family "My Jobs" keeps a stale "Hired" pill after the nanny resigns — CONFIRMED — `controllers/family_jobs_controller.dart:42-55` Failing use case: `_activeHires` is loaded once (no stream); after the nanny resigns, the family's My-Jobs still shows the job filled/"Hired" and won't surface it as reopen-able until a manual reload. Fix: reload hires on My-Jobs focus, or notify the family on resign.

[MAJOR] Two indistinguishable "Report" flows write to two different backends — CONFIRMED — `views/support/report_user_sheet.dart:82-97` (→ ticket) vs `views/support/report_problem_sheet.dart` (→ dispute) Failing use case: the same sheet title + 5-category vocabulary front two pipelines. Reporting the *same* nanny from her **profile** files a **support ticket** (`category:other`, reported-user id buried in free text, lands in Support); from the **chat flag** it files a **dispute** (structured `reportedUserId`/`category`, lands in My Reports with a chat). Users can't tell them apart, admin triages two queues, and the profile report loses the machine-readable target. Fix: unify on one model (route the profile "Report user" into the dispute pipeline, or give tickets structured `reportedUserId`/`category`) and differentiate the copy.

Minor

- **XR-1 · Every trial action double-notifies (trial trigger + chat-message trigger)** — `functions/src/triggers/chat.ts:5-35`. An offer sends the nanny two inbox rows + two pushes ("Trial offer received" + "New message"). *Fix:* suppress `onNewMessage` for trial system-bubble types.
- **XR-2 · "Trial completed" notifies only the family, with stale copy** — `functions/src/triggers/trial.ts:118-125`. On completion the family is told "Evaluate the nanny" (already done); the nanny gets no completion notice. *Fix:* drop/repurpose; notify the nanny of the outcome.
- **XR-3 · "Rate the app" fires after a *failed* trial** — `controllers/trial_controller.dart:532-534`, `rate_app_dialog.dart`. The prompt shows to the family right after a "Not this time" rejection (poor timing). *Fix:* gate the prompt to positive outcomes.
- **XR-4 · Contact reveal is family→nanny only, but the trial screen shows "Revealed" on the family card to the nanny** — `views/family/trial_screen.dart:383-390`, `functions/src/triggers/contact.ts:33-54`. During a trial the nanny sees the family marked "Revealed" implying she can call them, but she has no path to the family's number. *Fix:* give the nanny a reveal path during an active trial/hire, or drop the "Revealed" label in her view.
- **XR-5 · Family can close the trial outcome ("Hire") during an `accepted` (not-yet-started) trial, ungated by payment state** — `views/family/trial_screen.dart:474-547`. The nanny files a payment-issue dispute; the family still closes the trial as "hired" with no reconciliation, and the header hardcodes "Active" + a countdown regardless of real status. *Fix:* gate the outcome bar on `active`; reflect real status.

---

4. Backend / platform (breaks the mobile app)

These live in `functions/`, `firestore.rules`, or `firestore.indexes.json` but their failure surfaces in the app. (Backend items that primarily affect the admin dashboard are in the separate web report.)

Critical

[CRITICAL] Scheduled subscription jobs crash — missing `families.subscription` composite index — CONFIRMED — `functions/src/triggers/scheduled.ts:49-50,72-75`; `firestore.indexes.json` (absent)
Failing use case: `subscriptionExpiringReminder`/`subscriptionExpiredEnforcer` query `families` on `subscription.status (==/in) + subscription.endDate (<=)`, which needs a composite index that doesn't exist (the only status+endDate index is for an unused `subscriptions` collection-group). Both jobs fail `FAILED_PRECONDITION`, so **subscriptions never expire server-side and no 3-day reminders are sent**. Fix: add a `families` composite index `subscription.status ASC, subscription.endDate ASC`.

Major / security

[MAJOR · security] `syncMockSubscription` is a self-serve entitlement bypass, live because the project id contains "testing" — CONFIRMED — `functions/src/mockSubscription.ts:4-7`; honored at `firestore.rules:36-41`
Failing use case: any signed-in family can call the `syncMockSubscription` callable in `yhgc-testing`; `mockSubscriptionAllowed()` returns true on the `'testing'` substring, so it writes `subscription.status='active', mockDev:true` to the family's own doc via the admin SDK — instantly unlocking chat + contact for free (prod rules honor `mockDev`). Fix: gate on an explicit non-prod allow-list; ensure the real prod project id has no `testing`; strip `syncMockSubscription` + `familyHasMockDevAccess` from the prod deploy.

[MAJOR · security] Public admin bootstrap ships a hardcoded default password — CONFIRMED — `functions/src/utils/ensureFirstAdmin.ts:19`, `functions/src/bootstrapAdmin.ts:5-6` (detailed in the web report; also reachable from the app's ecosystem). Fix: require the env password, disable the endpoint after first use, rotate.

[MAJOR] Account-deletion cascade omits disputes, tickets, hires, profileViews, contactReveals — residual PII — CONFIRMED — `functions/src/triggers/delete.ts` Failing use case: once deletion can run at all (see §2 **Critical**), `delete.ts` cleans chats/trials/applications/shortlists/jobs/notifications/profiles/storage but leaves `disputes`(+messages, PII), `tickets`(+messages), `hires`, `profileViews`, `contactReveals`, `deletionAudits`, plus the user's messages inside other parties' threads. Fix: extend the cascade to those collections + subcollections.

Minor

- **BE-1 · Chat message create doesn't verify `senderId==uid`/`senderType`** — `firestore.rules:171-188`. A family with access can POST a message with `senderId=nannyId`/`senderType='nanny'`, fabricating a message the nanny then sees in-thread (the ticket/dispute rules DO pin this). *Fix:* require `incoming().senderId == request.auth.uid` + role-correct `senderType`.
- **BE-2 · Nanny can self-write server-owned `stats.*`** — `firestore.rules:67-74`. The self-update rule guards only `blocked`/`isVerified`/`status`; a modified client can inflate `stats.hiresCount`/`shortlists` (the app strips it, so vanity-only today). *Fix:* forbid client writes to `stats`/`profileScore`.
- **BE-3 · Aggregate counters are not idempotent** — `functions/src/triggers/stats.ts:24,74,87`, `trial.ts:28-31`. At-least-once redelivery double-runs `increment(1)`, drifting `shortlists`/`profileViews`/`hiresCount`. *Fix:* dedupe via an event-id marker, or make counters recomputable.
- **BE-4 · Applications/trials have no field-level write authorization** — `firestore.rules:110-113,201-204`. A nanny could flip her own application `declined→pending`, or a family set a trial `completed`. *Fix:* constrain allowed status transitions per role.
- **BE-5 · Emergency + reference phone numbers are sent to any family client** — `firestore.rules:58-60`, `nanny_model.dart:296,307`. An approved-nanny doc read by any family includes `emergencyPhone` + reference phones (not shown in UI, but transmitted to the client). *Fix:* move sensitive fields to a protected subcollection.
- **BE-6 · Delete cascade still queries the retired `reviews` collection** — `functions/src/triggers/delete.ts:102-115`. Dead code after the reviews-pipeline removal. *Fix:* remove.

---

5. Recurring root causes

1. **No realtime for hires / trials / employment.** Only chat threads stream; hires, trials, and employment cards are one-shot `.get()` loads. Any state change that doesn't mutate the thread doc (hire create/end, trial cancel, resign) never reaches the counterparty's open session. → drives the two hire Criticals + the stale My-Jobs pill.
2. **Family-oriented navigation/bindings reused for the nanny without role-branching.** `openThreadForNanny` matching by the nanny's own id, `Routes.trial` = `FamilyBinding`, "View trial" with no trialId, the trial "Family" card using `currentUser`. → drives the wrong-thread **Critical** + the trial cross-wire/binding Majors.
3. **Notification triggers assume a single actor, and several key events have no trigger at all.** `onTrialResponse` always blames the nanny/notifies the family; hire-create/hire-end/trial-cancel notify nobody. → drives most notification Majors.
4. **The monetization + trial-access layer is entirely mock in the shipping build.** No RevenueCat, expiry not re-checked, lockdown disabled, entitlement (`activeTrialNannyIds`) never granted on accept, and a self-serve bypass is live. → the biggest single cluster of release blockers.

---

6. Consolidated priority order (mobile)

Rank	Finding	Where
1	Delete Account fully broken (store blocker + PII)	§2 · `firebase_auth_service.dart:203`, `firestore.rules:52`, `delete.ts`
2	No real payment path / entire paywall is mock	§2 · `app_config.dart:13`, `firestore_subscription_service.dart:81`
3	Nanny "Message Family" → wrong/self conversation	§1 · `chat_controller.dart:285-325`
4	Nanny never told she's hired · hire-end invisible + reason overwrite	§3 · `stats.ts:80`, `firestore_hire_service.dart:42`
5	Scheduled subscription jobs crash (missing index) — subs never expire	§4 · `scheduled.ts:49`, `indexes.json`
6	Trial access never unlocks (`activeTrialNannyIds` on-end only) + `syncMockSubscription` bypass	§3/§4 · `trial.ts:113`, `mockSubscription.ts:4`
7	Nanny trial screen cross-wires (wrong trial/proof, wrong family name, FamilyBinding pollution)	§1 · `chat_screen.dart:770`, `routes.dart:132`
8	Subscription expiry not re-checked on resume + lockdown disabled	§2 · `subscription_controller.dart:63`, `chat_controller.dart:116`
9	Trial/hire notifications misfire (counter, cancel, completed, double-notify)	§3 · `trial.ts:51-125`, `chat.ts:5`
10	Match % divergence · dual report pipelines · second-job edit · Compare pair · availability control	§1–3
—	Then the **Minor** sweep (validation, i18n, resume-routing, rules-hardening)	§1–4

Every finding above is anchored to `file:line`. No app code was changed producing this audit.